Databases and Applications

Unit III

Pooja T S

Computer Applications

Databases and Applications

Unit III

Schema Design in MongoDB: Embedded vs Referenced

Pooja T S

Computer Applications

Databases and Applications

Why Schema Design Matters in MongoDB

PES UNIVERSITY

- **Schema design is critical even in NoSQL**
  - MongoDB allows flexible document structures
    - No fixed schema, but structure affects performance
    - Poor design increases query cost
    - Data duplication might grow uncontrollably
  - Designing based on *query patterns*
    - Embed when data is always read together
    - Reference when relationships grow large
    - Reduce round-trips for frequently accessed data

Databases and Applications

Schema Design in MongoDB – Overview

- MongoDB stores data as **documents** in **collections**, not rows in tables.
  - Documents are JSON-like structures (stored internally as BSON)
  - A collection can hold documents with flexible structure
- Schema design in MongoDB focuses on **how documents relate** to each other:
  - **Embedded** design – store related data inside a single document
  - **Referenced** design – store related data in separate collections and link them

Databases and Applications

Embedded Documents – Concept

PES UNIVERSITY

- In an **embedded schema**, related data is stored **inside** the same document.
  - One document contains all information needed for a use-case
  - No separate lookup is required in another collection
- Good when:
  - Relationship is **one-to-few** (limited number of related items)
  - Related data is **always accessed together** (high read locality)
  - Duplication is acceptable and does not grow unbounded

When to Use Embedded Schema

- **Choose embedding when:**
  - Data is accessed together 90% of the time
    - Example: blog post + comments
    - Example: invoice + line items
    - Example: student + list of enrolled courses
  - The related data has limited or predictable growth
    - Customer has 2–5 addresses only
    - Product has fixed set of specifications
    - Order rarely exceeds 10–20 items

Databases and Applications

Embedded Documents – Example

- Example: **Order document with embedded customer**

```
{
  _id:  ObjectId("..."),
  orderId:  "O101",
  orderDate:  ISODate("2025-01-10"),
  totalAmount:  2499,
  customer:  {
    customerId:  "C001",
    name:  "Amit Kumar",
    phone:  "98765...",
    city:  "Bengaluru"
  }
}
```

Databases and Applications

Referenced Documents – Concept

PES UNIVERSITY

- In a **referenced schema**, related data is stored in **separate documents**.
  - One document holds the main entity
  - Another document holds related entity, linked by an identifier
- Good when:
  - Relationship is **one-to-many** or **many-to-many**
  - Related data is reused across multiple documents
  - You want to avoid large document growth or duplication

Databases and Applications

When to Use Referenced Schema

- **Choose referencing when:**
  - Data grows without bound
    - Customer may place 500+ orders
    - Product may have thousands of reviews
    - A category links to thousands of SKUs
  - Data is reused across many documents
    - City list, department list, categories
    - Reduces duplication of standard information
    - Updates become easier and more consistent

Databases and Applications

Referenced Documents – Example

- **Customers** collection:

```
{
  _id:  ObjectId("..."),
  customerId:  "C001",
  name:  "Amit Kumar",
  phone:  "98765...",
  city:  "Bengaluru"
}
```

- **Orders** collection:

```
{
  _id:  ObjectId("..."),
  orderId:  "O101",
  orderDate:  ISODate("2025-01-10"),
  totalAmount:  2499,
  customerId:  "C001"
}
```

Embedded vs Referenced – Comparison

- **Embedded – Strengths**
  - Fastest reads (single document)
    - Optimised for read-heavy workloads
    - Minimises number of queries
    - Ideal for hierarchical data
  - Simple structure for client applications
    - Easier to serialize/deserialize
    - All data stays in one JSON object
    - No need for additional API calls

Databases and Applications

Embedded vs Referenced – High-Level View

- **Embedded** design:
  - Store related information **inside** one document
  - Fewer queries, excellent read performance for that document
  - Can lead to duplication if same embedded data repeats many times
- **Referenced** design:
  - Store related information in **separate** collections
  - Requires joining in application code (multiple queries)
  - Avoids duplication, suitable for shared data (e.g., same customer in many orders)

MongoDB Schema Design – Practical Rules

- **Rule 1: Model data based on application queries**
  - Schema follows the read patterns
    - Optimise for most frequent queries
    - Minimise number of lookups
    - Reduce joins at application layer
  - **Rule 2: Consider update frequency**
    - Frequently updated fields → use reference
    - Rarely updated fields → embed
    - High-write workloads prefer smaller docs

Databases and Applications

MongoDB Schema Design – Practical Rules

PES UNIVERSITY

- **Rule 3: Think about document size**
  - MongoDB document limit is 16 MB
    - Large arrays → move to separate collection
    - Big logs/history → reference
    - Avoid uncontrolled growth within documents
  - **Rule 4: Think about cardinality**
    - Low cardinality → embed
    - High cardinality → reference
    - Many-to-many → always reference

Customer–Order Example – Embedded Schema

- **Single collection:** `orders` with embedded customer

Collection: `orders`

```
_id: ObjectId("...")
orderId: "O101"
orderDate: ISODate("2025-01-10")
totalAmount: 2499
customer: {
  customerId: "C001"
  name: "Amit Kumar"
  phone: "98765..."
}
```

- Interpretation:
  - All order + customer details are stored in a **single document**
  - Best for cases where a customer has a small to moderate number of orders

Databases and Applications

Customer–Order Example – Referenced Schema

- **Two collections:** `customers` and `orders`

Collection: `customers`

```
_id:  ObjectId("...")
customerId:  "C001"
name:  "Amit Kumar"
phone:  "98765..."
city:  "Bengaluru"
```

Collection: `orders`

```
_id:  ObjectId("...")
orderId:  "O101"
orderDate:ISO("2025-01-10")
totalAmount:  2499
customerId:  "C001"
```

`customerId` reference: `"C001"` → links customers → orders

Databases and Applications

When to Embed vs When to Reference

- **Prefer Embedded** when:
  - You read the parent and child data **together** most of the time
  - Child collection size is small (one-to-few)
  - Data duplication does not create maintenance issues
- **Prefer Referenced** when:
  - A child (e.g., customer) is shared across many parents (many orders)
  - You want a single, central source of truth for the entity
  - Documents would otherwise become very large or frequently updated

Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 391